Because each process in Elixir has its own memory heap and stack, each process performs its own garbage collection. The binary heap that was mentioned in the previous section is globally shared, so there's a global collection process to handle it. However, it's relatively lightweight because the binary heap uses reference-counted binaries.

Each garbage collection process runs fast because it deals with less memory, and it happens on a cycle according to how much that process is churning data. If the process is frequently processing messages or is running out of memory, it will experience a collection cycle more often than a process that isn't doing much.

But you have to be cautious of a hidden danger. Sometimes long-lived processes can take up more memory than they need, but they won't undergo a garbage collection cycle because they aren't active enough to kick one off. In this situation, a process can take up more memory than it needs for a long period of time. Let's create an artificial example:

```
$ iex -S mix
iex> pid = Examples.Spawn.Infinite.start()
iex> Enum.each(1..1000, & send(pid, {:add, &1, &1}))

iex> send(pid, :memory)
I am using 62816 bytes
```

Your numbers may vary here, but you should expect to see that this number is higher than the 2.6 KB that it started out as. This doesn't immediately make sense because our process has no state, and it has processed all of its messages. So, why is it taking up thirty times more memory?

The issue here is that the process mailbox lives on the heap of the process. As we inundated it with messages, it had to allocate more memory to hold those messages. The garbage collection process only occurs based on running out of memory or processing a given number of reductions—neither of which is occurring.

We can manually trigger garbage collection with :erlang.garbage_collect(pid). Once you do this and query the process memory, you'll see that it's back to its starting size.

```
iex> :erlang.garbage_collect(pid)
iex> send(pid, :memory)
I am using 2608 bytes
```

The number of long-lived processes is usually small enough that this doesn't matter. But, if it becomes a problem, then look at the ERL_FULLSWEEP_AFTER system variable and set it to a number like 20. This causes garbage collection